

Orthographic Nativization as Rule-Based Rewrite Cascades

Erin Gabrielle Chua, Zhean Robby Ganituen*, Justin Ethan Ching, Jaztin Jacob Jimenez, Nathaniel Oco

De La Salle University

Manila, Philippines

*zhean_robbly_ganituen@dlsu.edu.ph

Abstract—Orthographic nativization, generating standardized written forms of loanwords, remains understudied computationally despite its importance for text normalization and multilingual NLP. We present a rule-based approach for orthographic nativization of English loanwords into Filipino. The system operates through multiple stages: graphemic tokenization, priority-ordered rewrite rules, and grapheme-to-phoneme lookup. Rules are hand-authored from orthographic guidelines and phonological sources, enabling deployment without parallel training corpora. We evaluate on a newly constructed gold standard of 2,319 English–Filipino loanword pairs. The proposed method achieves 5.14% error rate under a 5-vowel and 4.05% under a 3-vowel evaluation. The system outperforms a regular expressions and zero-shot prompting of a large language model given the same mapping rules. Error analysis reveals that most remaining errors stem from schwa ambiguity in unstressed syllables, glide insertion, and the quality of the phonetic transcription. The linguistically grounded approach demonstrates that prescriptive rules can be operationalized into effective orthographic nativization systems for low-resource languages.

Index Terms—orthographic nativization, loanword adaptation, Filipino, rule-based systems, low-resource NLP

I. INTRODUCTION

Borrowed words rarely enter a language unchanged. When English loanwords are adopted into Filipino, they must be adapted to conform to Filipino’s orthographic conventions, a process known as *orthographic nativization*. This computational task requires generating the standardized written form of a loanword according to the recipient language’s spelling rules. For example, “computer” becomes “kompyuter”, and “basketball” becomes “basketbol”. Despite its practical importance for text normalization, spell-checking, and corpus construction in multilingual natural language processing, orthographic nativization has received considerably less computational attention than the well-studied problems of phonological and morphological adaptation [?], [?], [?], [?].

Filipino provides an ideal testbed for orthographic nativization. The language borrows extensively from English across education, medicine, commerce, and public administration [?], [?], and official nativization guidelines are codified in the prescriptive manual by the Komisyon sa Wikang Filipino [?]. These properties enable systematic evaluation against standardized orthographic norms. Furthermore, Filipino is typologically distinct from English in phonological inventory, syllable structure, and orthographic

depth [?], [?], [?]. This typological distance ensures that the proposed system must handle substantial graphemic and phonotactic transformations, providing a rigorous test of the approach.

Our method draws on prior work in computational loanword modeling while addressing a distinct generation problem. Finite-State Transducer (FST) with Optimality-Theoretic constraint ranking have successfully modeled the morpho-phonological transformations of Arabic loanwords entering Swahili for bilingual lexicon generation [?], and similar FST approaches have captured much of the phonological adaptation from English into Japanese [?]. Furthermore, rule-based methodologies have been explored for adjacent tasks like machine transliteration and generation of English respellings [?], [?]. More broadly, nativization in loanword phonology has been shown to be systematic and rule-governed across typologically diverse languages [?]. However, these studies focus on modeling phonological patterns (the abstract sound structure of adaptation) rather than orthographic generation. As documented in [?], [?], phonological and orthographic nativization are not equivalent: a phonological model produces an abstract pronunciation representation, while an orthographic nativization system must produce a concrete character string that obeys the recipient’s spelling rules.

The proposed system performs orthographic nativization through a four-stage pipeline: graphemic tokenization, rule-based grapheme mapping, phonetic resolution via Grapheme-to-Phoneme (G2P) lookup, and output normalization. Rules are hand-authored from prescriptive guidelines [?] and linguistic descriptions of Filipino phonology [?], [?], [?], [?], [?], making the approach transparent and linguistically grounded. By deriving rewrites from existing linguistic resources rather than requiring parallel training corpora, the method is particularly suited to low-resource scenarios where annotated loanword data are unavailable.

Our primary contributions are as follows:

- **Linguistic rules as a rule-based system:** We demonstrate that prescriptive linguistic rules can be operationalized into an effective rewrite-cascade pipeline without large parallel corpora, achieving 5.14% character error rate under strict evaluation and 4.05% when accounting for Filipino’s vowel variation.

- **Empirical Case Study and Dataset:** We provide an empirical case study on English–Filipino borrowing using a newly constructed gold standard of 2,319 loanword pairs, identifying systematic error patterns and validating the utility of phonetic disambiguation for vowel adaptation.

II. FILIPINO ORTHOGRAPHY AND SYSTEM DESIGN

The orthographic properties of Filipino directly motivate how our system handles phonetic vowel resolution, consonant mapping, and syllable repair. This section describes these properties and explains how each shapes the corresponding stage of the system.

A. Vowel Spelling and Phonetic Resolution

Filipino orthography uses five vowel letters (“a”, “e”, “i”, “o”, “u”), each with a stable sound. Because English’s vowel inventory is larger, multiple English vowels must collapse onto a single Filipino letter: English /æ/ (“cat”) and /ɑ:/ (“father”) are both written “a”. This many-to-one relation creates an ambiguity that English spelling alone cannot resolve, since the same letter “a” corresponds to /eɪ/ in “make” but /æ/ in “cat.” Compounding this, English is stress-timed, so unstressed vowels reduce to schwa /ə/, a distinction that orthography does not encode at all. Resolving which English vowel sound underlies a given letter therefore requires information beyond spelling: the pronunciation itself.

The second property is one of spelling convention. The vowels (“e”, “i”, “y”) and (“o”, “u”) are widely treated as interchangeable in many lexical contexts, a convention rooted in the historical allophonic status of these pairs in Filipino [?]. Native speakers often accept either variant without perceiving an error, and this latitude is especially common in borrowed vocabulary, where prescriptive sources do not always fix a unique choice.

B. Consonant Letter Correspondences and Context-Sensitive Rules

For the consonant letters native to the Filipino alphabet [?], English–Filipino spelling correspondences are largely predictable one-to-one mappings. However, English loanwords frequently contain sounds with no native Filipino letter, notably /ʃ/, /tʃ/, and /dʒ/. Prescriptive guidelines [?], [?] supply standard written forms: /tʃ/ is written “ts”, /dʒ/ as “dy”, and /ʃ/ as “s” or “sy”. Because such correspondences are fixed, they can be applied as invariant substitutions.

However, not all consonant graphemes are invariant. Some are nativized depending on adjacent graphemes.

C. Spelling Repairs and Output Normalization

Filipino spelling strongly favours syllables of the form consonant-vowel or consonant-vowel-consonant [?], [?]. When a direct mapping would yield a consonant cluster that violates this template, prescriptive convention inserts a vowel letter to break the illegal sequence [?]. Enforcing this constraint requires a repair pass over the mapped output.

Algorithm 1 Orthographic Nativization

Require: English loanword w

Ensure: Filipino orthographic form

```

1:  $G \leftarrow \text{TOKENIZE}(w)$ ;  $P \leftarrow \text{G2P}(w)$ 
2:  $R \leftarrow \langle \rangle$ ;  $i \leftarrow 1$ 
3: while  $i \leq |G|$  do
4:   for  $\rho \in \langle \text{SENSITIVE}, \text{PHONETIC}, \text{FREE} \rangle$  do
5:     if  $(r, c) \leftarrow \rho(G, P, i)$  succeeds then
6:        $R \leftarrow R \cdot r$ ;  $i \leftarrow i + c$ ; break
7:     end if
8:   end for
9: end while
10: return  $\text{NORMALIZE}(R, P)$ 
```

Algorithm 2 Graphemic Tokenization

```

1: procedure  $\text{TOKENIZE}(w)$ 
2:    $G \leftarrow \langle \rangle$ ;  $i \leftarrow 1$ 
3:   while  $i \leq |w|$  do
4:      $\ell \leftarrow \max\{k \in \{4, 3, 2\} : w[i:i+k-1] \in M_k\}$ ,
5:     or 1 if no multigraph matches
6:      $G \leftarrow G \cdot w[i:i+\ell-1]$ ;  $i \leftarrow i + \ell$ 
7:   end while
8:   return  $G$ 
9: end procedure
```

III. PROPOSED METHOD

The system performs orthographic nativization of English loanwords through a four-stage pipeline: graphemic tokenization, rule-based grapheme mapping, phonetic resolution via G2P lookup for ambiguous vowels, and output normalization. A single priority-ordered rewrite cascade, applied left to right, carries out both grapheme mapping and phonetic resolution; the G2P transcription it consults is computed once, up front. The input is the English orthographic form; the output is the Filipino form prescribed by [?]. The linguistic properties of section II determine the content of each stage. Figure 1 gives the architecture, and algorithm 1 states the top-level control flow.

Every rule returns a pair (r, c) : a (possibly empty) *sequence* of Filipino graphemes and the number of English graphemes *consumed*. Position advances by c so a single rule may rewrite a multi-grapheme span. For example, “ck” $\mapsto$ “k” consumes two.

A. Graphemic Tokenization

English’s deep orthography contains multi-letter sequences that behave as single units, so the input is first segmented into graphemes. The system recognizes digraphs (e.g., “ph”, “ch”), trigraphs (e.g., “ore”, “tch”), and tetragraphs (“augh”, “ough”, “eigh”). Writing M_k for the inventory of length k , tokenization is greedy longest-match (algorithm 2): at each position the longest matching multigraph is emitted, falling back to a single character. Thus “straight” segments as “s-t-r-a-igh-t” and “phishing” as “ph-i-sh-i-ng”.

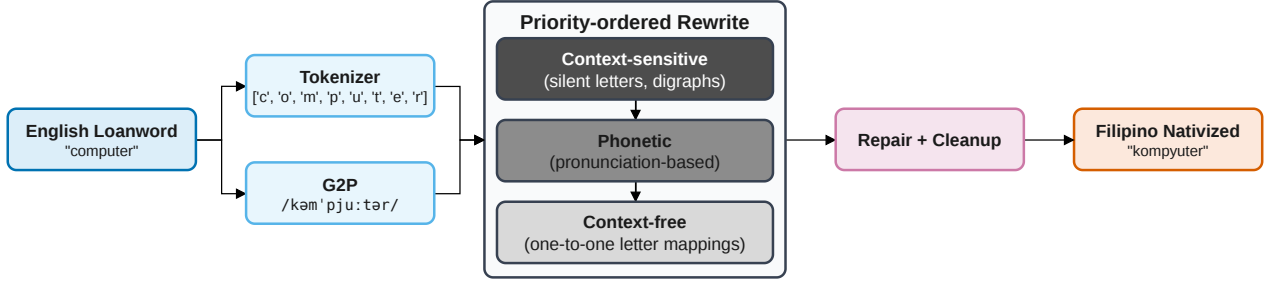


Fig. 1. Orthographic nativization pipeline. An English loanword is tokenized and passed through a priority-ordered rewrite cascade (context-sensitive rules, then phonetic rules, then context-free mappings) followed by repair and cleanup to produce the Filipino nativized form (e.g., *computer* $\rightarrow$ *kompyuter*).

B. Rewrite Cascade

Graphemes are mapped to their Filipino equivalents through the priority-ordered cascade of algorithm 1, taking the first rule that succeeds: context-sensitive orthographic rules, phonetic resolution for ambiguous vowels, and context-free orthographic rules as the default.

a) *Context-free rules*: Many English consonants bear an invariant relation to their nativized form, so mapping reduces to direct substitution independent of context: “ph” $\mapsto$ “f”, “th” $\mapsto$ “t”, “ee” $\mapsto$ “i”, “oo” $\mapsto$ “u”, “kn” $\mapsto$ “n”, “wr” $\mapsto$ “r”, and “dge” $\mapsto$ “j”. The borrowed letters “v”, “z”, “j” and the digraph “sh” are retained rather than folded onto native counterparts, following the extended alphabet of section II. A context-free mapping is defined for every grapheme, including those normally handled contextually; the cascade can therefore always fall back on one, and repeated letters can be resolved by it.

b) *Context-sensitive rules*: The rewriting rules are organized by the specific letter or sequence being evaluated. The system checks the current letter and its neighbors to decide how it should be replaced. Example cases include:

- Repeated letters: a maximal run of identical graphemes is consumed whole and emits the context-free replacement of the first. The exception is “cc” before “e”, “i”, “y”, or “ee”, which the “c” rules take instead so that the two letters may map differently: “accept” $\mapsto$ “aksept”, “success” $\mapsto$ “sakses”.
- Cases for “c”: “c” $\cdot$ (“e” | “i” | “y” | “ee”) $\mapsto$ “s”; “ck” $\mapsto$ “k” and “cch” $\mapsto$ “k”, both consuming two; otherwise “c” $\mapsto$ “k”.
- Silent “e”: a word-final “e” is dropped unconditionally, regardless of the preceding grapheme.

C. Phonetic Resolution

Vowels are the difficult case. A single English vowel grapheme can spell several phonemes, and the five-vowel Filipino system collapses those distinctions; worse, unstressed vowels reduce to a schwa that the spelling does not mark. We call a grapheme *ambiguous* when its Filipino form cannot be read off the spelling. When a context-sensitive rule does not settle such a grapheme, the system resolves it from the pronunciation (algorithm 3).

Algorithm 3 Phonetic Resolution

```

1: procedure PHONETIC( $G, P, i$ )
2:    $t \leftarrow G[i]$ 
3:   if  $t$  is not an ambiguous vowel then return fail
4:   end if
5:   if  $t$  continues a vowel cluster then return ( $\langle \rangle, 1$ )
6:   end if
7:    $p \leftarrow$  vowel phoneme of  $t$  in  $P$ , or fail if none
8:   if  $p$  is r-colored then
9:     return ( $\langle t \rangle, 1$ ) if medial, else ( $\langle \text{“e”}, \text{“r”} \rangle, 1$ )
10:  end if
11:  if  $p = /ə/$  then
12:    return ( $\langle \text{“o”} \rangle, 1$ ) if word-final and  $t \in \{\text{“o”}, \text{“u”}\}$ ;
13:    ( $\langle \text{“e”} \rangle, 1$ ) if word-final; fail otherwise
14:  end if
15:   $(r, d) \leftarrow \text{P2G}(p)$ 
16:  if  $r = \langle \text{“a”} \rangle$  and  $t \in \{\text{“o”}, \text{“u”}\}$  then
17:     $r \leftarrow \langle t \rangle$ 
18:  end if
19:  return ( $r, \text{CONSUMED}(G, i, d)$ )
20: end procedure

```

The function P2G maps a single phoneme to its Filipino spelling. Diphthongs become “ey”, “ay”, “aw”, or “oy”. The one exception is /ou/, which becomes a single “o”. For instance, the /e/ in “make” gives “meyk”. A lax monophthong takes the nearest Filipino simple vowel, so the /æ/ of “cat” gives “a”. When that nearest vowel is “a” but the written letter is “o” or “u”, the written letter is kept instead. R-colored vowels and word-final schwas are spelled directly. A medial r-colored vowel, one followed by “r” and at least one further grapheme, keeps its written letter; elsewhere it is spelled “er”. A word-final schwa is spelled “o” after a written “o” or “u”, and “e” after any other letter. A schwa in any other position fails, falling through to the context-free rules by design.

D. Output Normalization

NORMALIZE cleans up artifacts left where independent rules meet at a boundary. First, adjacent identical graphemes

emitted by different rules are collapsed to one: a rule emitting “d” directly before another emits “dy” produces “ddy”, which collapses to “dy”. Second, phonotactic cluster repair inserts an epenthetic “o” to break up disallowed word-final consonant-liquid clusters derived from English schwas.

IV. EXPERIMENTAL SETUP

The source code and gold standard are publicly available at <https://github.com/Mango-Cats/tagabaybay>. The rules are implemented in Rust; the G2P backend uses the phonemizer library [?], which wraps eSpeak-NG.

A. Gold Standard

The gold standard corpus comprises 2,319 English–Filipino loanword pairs, drawn from a list English words commonly used in Filipino everyday speech (1,858 pairs) and pharmaceutical terminology (461 pairs). Three native Filipino mesolectal speakers, familiar with the KWF manual beforehand, independently annotated the full word list to produce nativized forms. Annotators applied the spelling conventions prescribed in [?], [?]; where the manual did not prescribe a specific nativization, they consulted [?], [?] to determine the most plausible form under the five-vowel system.

Before adjudication, majority-match agreement was 92%. The remaining items, where no two annotators agreed, were resolved through discussion among all three annotators, and the corpus was finalized accordingly.

B. Evaluation Metrics

The primary evaluation metric is Character Error Rate (CER), computed as the Levenshtein edit distance between the system output and the gold standard reference, normalized by the total character count of the reference across the entire corpus:

$$\text{CER} = \frac{\sum_i \text{LEVENSHTEIN}(\hat{y}_i, y_i)}{\sum_i |y_i|} \times 100\%$$

where $\hat{y}_i$ is the system’s nativized form of the i -th word, y_i is the gold standard reference, $\text{LEVENSHTEIN}(\cdot, \cdot)$ denotes Levenshtein distance, and $|y_i|$ is the character length of the reference. CER is preferred over word-level accuracy because nativization errors are often minor, and a word-level metric would treat a one-character discrepancy as harshly as a completely incorrect output.

Additionally, because (“e”, “i”, “y”) and (“o”, “u”) are interchangeable in Filipino [?], we report CER under additional conditions treating these pairs as equivalent.

C. Comparison Systems

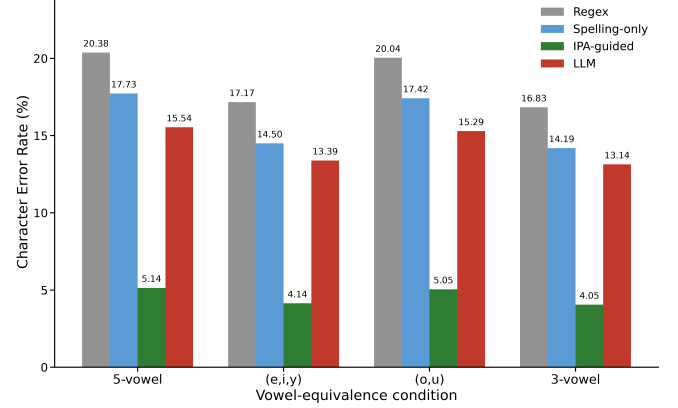
We evaluate three additional systems. First, a regex-based system applies all proposed rules that allow for direct pattern substitution as a flat, unordered sequence, excluding rules that require graphemic tokenization, priority ordering, or phonetic disambiguation. Second, a spelling-only ablation runs the full proposed system but disables phonetic vowel

resolution to isolate the impact of G2P-based disambiguation. Finally, we also evaluate an LLM. We use Claude Haiku 4.5 to process each word individually. The prompt provides examples alongside a bulleted list of the KWF mapping rules without any further prompt engineering.

V. RESULTS AND DISCUSSION

The system was evaluated on 2,319 English–Filipino loanword pairs. Figure 2 reports overall CER under four vowel-equivalence conditions.

Fig. 2. Overall CER (%) under four vowel-equivalence conditions.



The IPA-guided system achieved 5.14% CER under strict five-vowel evaluation, a 12.59-point reduction from the 17.73% spelling-only ablation and a 15.24-point reduction from the 20.38% regex-based system. The LLM achieved 15.54% CER, outperforming the regex-based system but underperforming the spelling-only ablation. This result indicates that even with explicit nativization rules in the prompt, an LLM cannot reliably apply them at the character level, and that the rewrite cascade substantially outperforms prompting-based approaches for this task. Under (“e”, “i”, “y”) equivalence, CER fell to 4.14%; under (“o”, “u”) equivalence it yielded 5.05%; the combination of both produced 4.05%. The larger gain from (“e”, “i”, “y”) equivalence reflects two overlapping sources of vowel ambiguity: schwa resolution and (“i”, “y”) alternation in glide contexts. The marginal effect of (“o”, “u”) equivalence confirms that this alternation is rare in the evaluation data.

A. Error Analysis

Manual inspection of the 856 character-level edits under 3-vowel scoring reveals four dominant failure modes.

1) *Unstressed Vowel Ambiguity*: Because English heavily relies on the unstressed schwa sound, which lacks a direct phonological equivalent in Filipino, nativization often forces an arbitrary mapping to either “e”, “i”, or other vowels based on auditory approximation. Consequently, schwa resolution is the largest error source for the system.

Even with phonetic resolution, the system still fails on some of these mappings because the ambiguity is structural. The G2P mapping can identify the unstressed vowel, but

accurately predicting which specific orthographic vowel a Filipino speaker will substitute requires lexical stress information that the pipeline cannot fully resolve on its own.

Under 3-vowel evaluation, these vowel-to-vowel substitutions dominate the error rates: “a” $\mapsto$ “o” accounts for 46 errors, “e” $\mapsto$ “a” for 37, “o” $\mapsto$ “a” for 27, and “a” $\mapsto$ “e” for 23, which total over 133 vowel substitutions. The (“a”, “o”) contrast persists even after (“e”, “i”, “y”) and (“o”, “u”) are collapsed. By contrast, the marginal effect of (“o”, “u”) equivalence confirms that this particular alternation is rare in the evaluation data, likely because English “o” and “u” sounds map more predictably into Filipino’s vowel space.

2) *Incomplete Glide Insertion*: Filipino phonotactics requires glides /w/ and /j/ to break vowel hiatus. The current rules miss 42 “w”-insertion and 40 “y”-insertion sites under 5-vowel evaluation. “tiara” maps to “tiara” instead of the expected “tiyara”. Additional context conditions for vowel-initial suffixes and word-final sequences would close these gaps.

3) *G2P Propagation Errors*: Some failures originate in the upstream G2P module rather than in the rewrite rules. An incorrect IPA transcription directly affects the output: “chlorpromazine” yields “klorop~~o~~ramasin” instead of “klorpromasin”. 40 of the edits trace to incorrect G2P transcriptions, and these concentrate in pharmaceutical terms and long compound words where eSpeak-NG’s accuracy degrades. A specialized pronunciation lexicon would directly reduce this error class.

B. Limitations

The rules derive from prescriptive sources and may not capture informal nativization patterns. Furthermore, the rule set is Filipino-specific, though the pipeline architecture, multigraph tokenizer, and rewrite cascade are language-general; only the rule inventory, phoneme-to-grapheme mappings, and normalization repairs require re-engineering per target language. Finally, G2P accuracy bounds overall performance, as phonemization errors propagate directly to the output.

VI. CONCLUSION

We presented a rule-based rewrite cascade for orthographic nativization of English loanwords into Filipino. The system achieved 5.14% CER under 5-vowel evaluation and 4.05% under 3-vowel evaluation, without parallel training data. Error analysis identifies schwa-derived vowel ambiguity as the primary remaining error source. Incorporation of lexical stress information or a hybrid rule-and-data approach could yield further gains. The framework’s transparency makes it suitable for low-resource scenarios where annotated loanword corpora are unavailable. Future work will extend the approach to other language pairs and evaluate on informal register text.

ACKNOWLEDGMENTS

The authors thank Clarence Ivan Ang, Clive Jarel Ang, and Roan Cedric Campo for their contributions to an earlier prototype of the system.